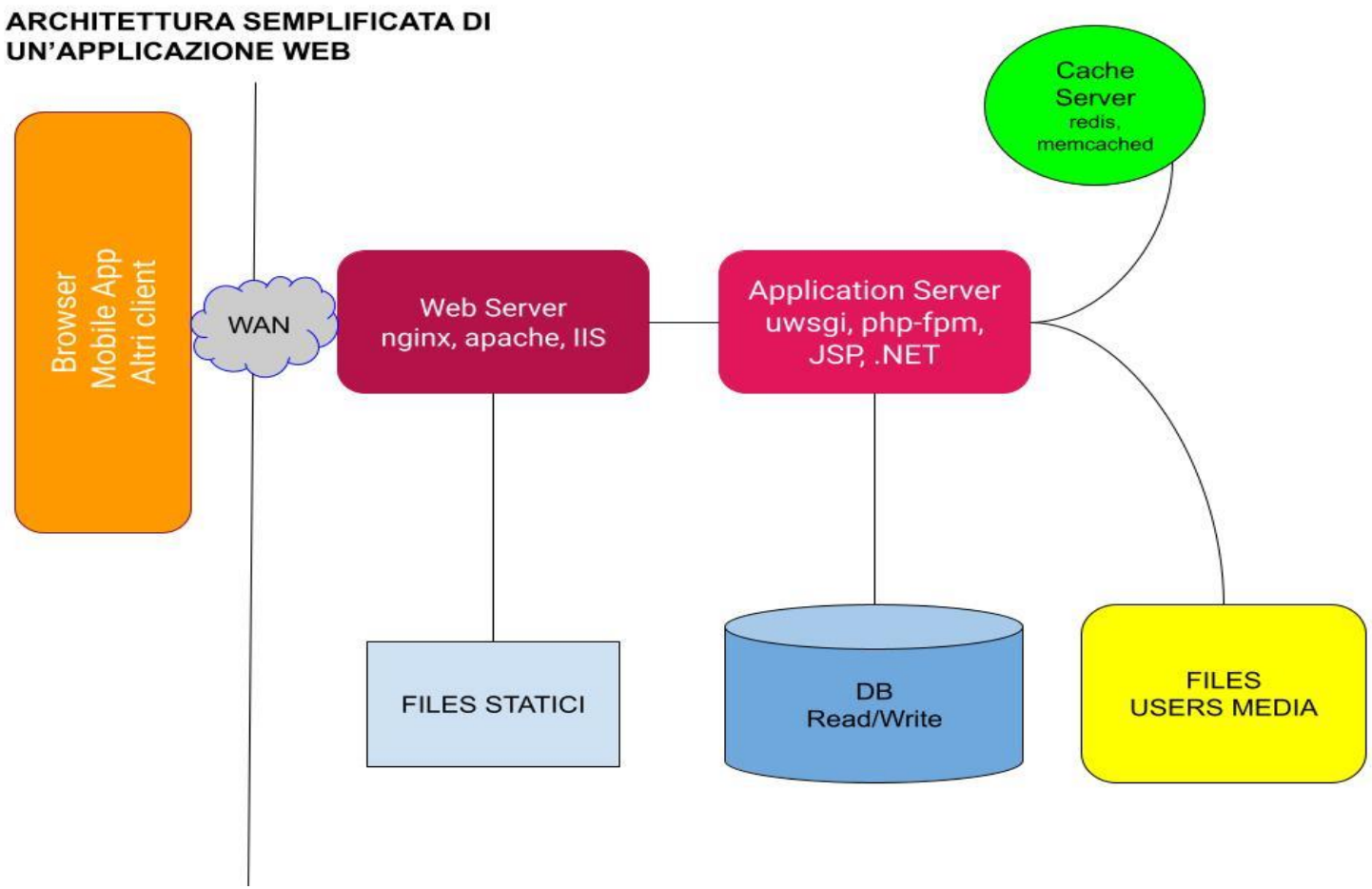
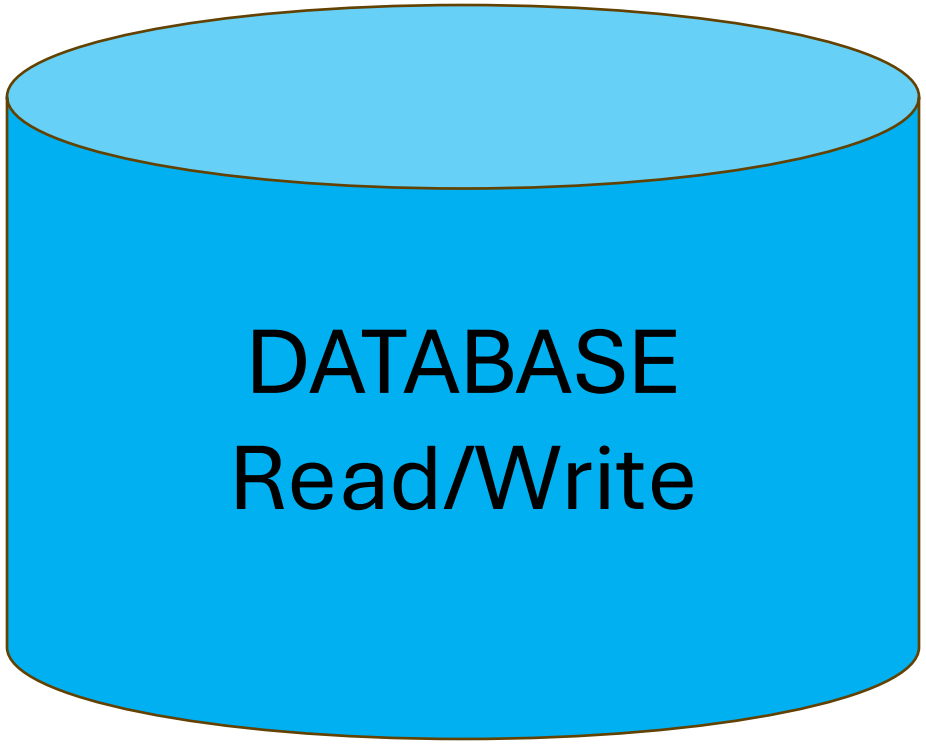
Architettura Cloud @ITS

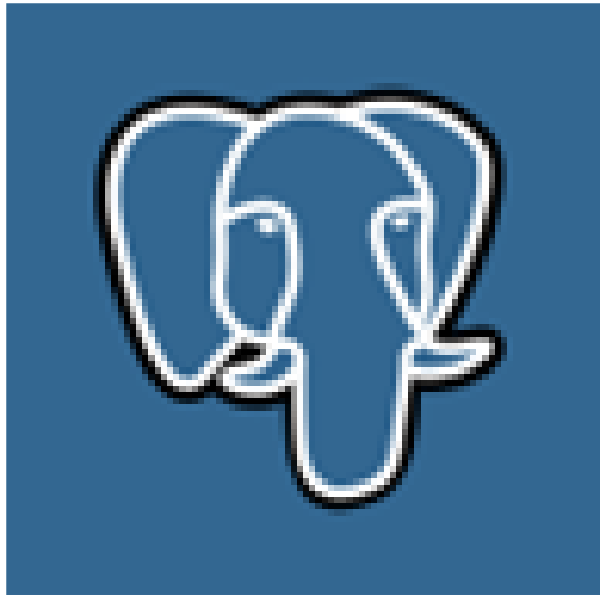
12 marzo 2026 – COMPOSE, prof. Luca Ferroni

Obiettivi lezione 3: "COMPOSE"

- Database server POSTGRES
- Cache server REDIS
- Application server
- Docker compose reale


ARCHITETTURA SEMPLIFICATA DI UN'APPLICAZIONE WEB
Browser
Mobile App
Altri client
WAN
Web Server
nginx, apache, IIS
Application Server
uwsgi, php-fpm,
JSP, .NET
Cache
Server
redis,
memcached
FILES STATICI
DB
Read/Write
FILES
USERS MEDIA





PostgreSQL

Alternative: MariaDB, MySQL, Oracle, SQL Server

Compose RDBMS PostgreSQL

```
feroda@paco:~/play-cloud-app$ cp env_dist .env
feroda@paco:~/play-cloud-app$ cat .env
POSTGRES_DB=changeme
POSTGRES_USER=changeme
POSTGRES_PASSWORD=changeme
```

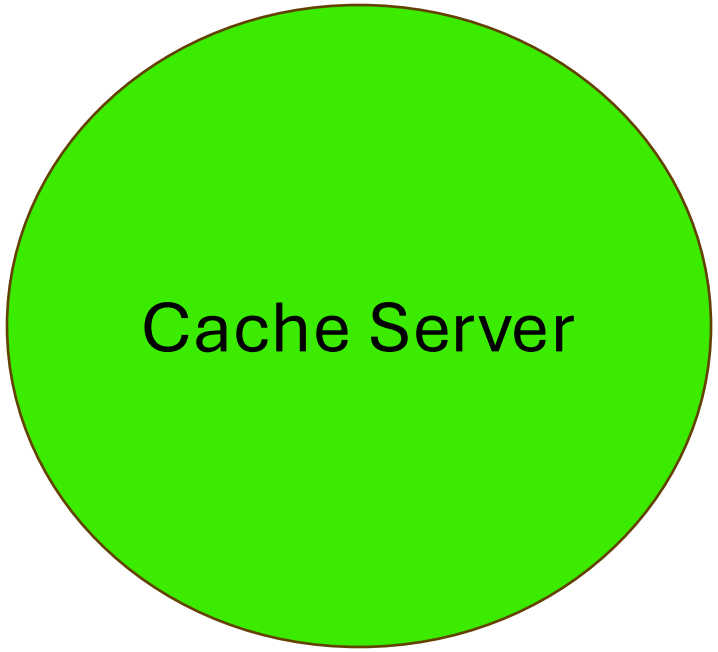
```
feroda@paco:~/play-cloud-app$ cat compose-db.yml
services:

  # Database PostgreSQL
  db:
    image: postgres:18-alpine
    env_file: .env
    volumes:
      - pg_data:/var/lib/postgresql/data
      - ./db:/db
    networks:
      - appnet
    restart: unless-stopped

networks:
    - appnet:

volumes:
  - pg_data: {}
```

Try it!

feroda@paco:~/play-cloud-app$ **docker-compose -f compose-db.yml up -d**

```
[+] Running 10/10
 ✔ db Pulled
   ✔ 589002ba0eae Pull complete
   ✔ a92c4c6bf7bf Pull complete
   ✔ 36af16c7d2ce Pull complete
   ✔ 455d094c943c Pull complete
   ✔ eb134dfb2af4 Pull complete
   ✔ 946b64497533 Pull complete
   ✔ 7fb0a4f2ca3f Pull complete
   ✔ b88294e2a4c6 Pull complete
   ✔ 51718cc743bc Pull complete
[+] Running 3/3
 ✔ Network my-django-dockerapp_appnet     Created
 ✔ Volume "my-django-dockerapp_pg_data"   Created
 ✔ Container my-django-dockerapp-db-1     Started
```

feroda@paco:~/play-cloud-app$ **docker-compose -f compose-db.yml exec db /bin/sh**
/ # psql
psql: error: connection to server on socket "/var/run/postgresql/.s.PGSQL.5432" failed: FATAL: role "root" does not exist
/ # psql -U $POSTGRES_USER
psql (18.3) Type "help" for help.
changeme=# **CREATE TABLE demo (id SERIAL, name VARCHAR(32), value VARCHAR(256));**
CREATE TABLE
changeme=# **INSERT INTO demo (name, value) VALUES ('cognome', 'Ferroni');**
INSERT 0 1
changeme=# **\q**
/ # exit
feroda@paco:~/play-cloud-app$ **docker-compose -f compose-db.yml stop**



Redis

Compose Cache server Redis

feroda@paco:~/play-cloud-app$ cat compose-redis.yml

```
services:

  # Redis Cache Server
  redis:
    image: redis:8.6.1-alpine
    sysctls:
      net.core.somaxconn: '511'
    volumes:
      - redis_data:/data
    networks:
      - appnet
    restart: unless-stopped

networks:
    - appnet:

volumes:
  - redis_data: {}
```

Se vuoi contattarlo dall'host locale aggiungi

```
ports:
  - 127.0.0.1:6379:6379
```

Try it!

feroda@paco:~/play-cloud-app$ **docker-compose -f compose-redis.yml up -d**

feroda@paco:~/play-cloud-app$ **docker-compose -f compose-redis.yml exec redis /bin/sh**
/data # redis-cli

127.0.0.1:6379> KEYS '*' (empty array) 127.0.0.1:6379> SET 'name' 'Luca' OK 127.0.0.1:6379> SET 'surname' 'Ferroni' OK 127.0.0.1:6379> KEYS '*' 1) "name" 2) "surname"	127.0.0.1:6379> GET name "Luca" 127.0.0.1:6379> MGET name surname "Luca" "Ferroni" 127.0.0.1:6379> exit

/data # exit
feroda@paco:~/play-cloud-app$ **docker-compose -f compose-redis.yml stop**

Application server

Alternative: PHP-FPM per php, JSP per Java, NodeJS per Javascript, ASP.NET per C#

Application servers

- ✅ **uWSGI** è un application server che esegue applicazioni Python (e altri linguaggi) esponendole tramite protocolli web come WSGI/HTTP

- ✅ **PHP-FPM** (FastCGI Process Manager) è un application server che esegue applicazioni PHP gestendo processi ottimizzati ed esponendole tramite il protocollo FastCGI

- ✅ **JSP/Servlet, ad es: Tomcat** è un server che esegue applicazioni Java trasformando pagine JSP in servlet ed esponendole tramite HTTP

- ✅ **ASP.NET** è un server che esegue applicazioni C# (ASP.NET) e le espone come servizi web tramite HTTP

- ✅ **Node.js** è un runtime e application server che esegue applicazioni JavaScript lato server ed espone servizi web tramite HTTP

Immagine Docker non disponibile?

...

La creiamo noi!

uwsgi/Dockerfile

```
FROM python:3.14.3-slim-trixie

ENV PYTHONUNBUFFERED 1
RUN apt-get update && apt-get install -y lsb-release wget build-essential gettext

COPY requirements.txt /
RUN pip install --no-cache-dir -r /requirements.txt
WORKDIR /app

CMD ["python"]
```

uwsgi/requirements.txt

```
uwsgi
```

Compose uWSGI

```
feroda@paco:~/play-cloud-app$ cat compose-uwsgi.yml
services:

  web:
    build: ./uwsgi/.
    volumes:
        - ./uwsgi:/app
    command: uwsgi --ini uwsgi.ini
    ports:
        - 10001:8001
    stop_signal: SIGINT
    restart: unless-stopped
    networks:
       - appnet

networks:
      appnet:
```

Servire la app hello.py con uWSGI

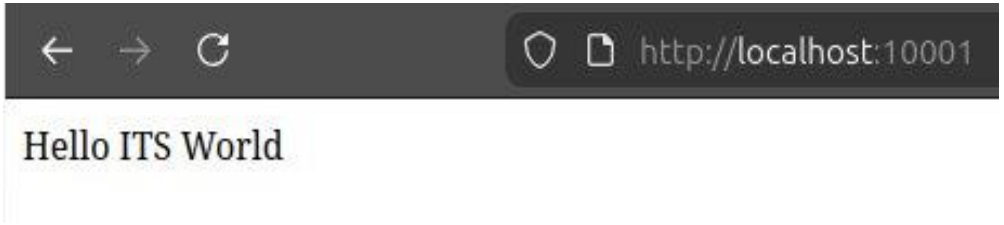
uwsgi/hello.py

```
def application(env, start_response):
    start_response('200 OK', [('Content-Type','text/html')])
    return [b"Hello ITS World"]
```

docker-compose -f compose-uwsgi.yml up –d

Invece di un comando simile a…

```
docker run --rm -p 10001:8001 --name uwsgi-10001 -v ./uwsgi:/app/
befair/uwsgi-python uwsgi --http :8001 --chdir /app/ --module
hello:application
```



Application server

Alternative: PHP-FPM per php, JSP per Java, NodeJS per Javascript, ASP.NET per C#

web/Dockerfile

```
FROM python:3.14.3-slim-trixie

ENV PYTHONUNBUFFERED 1
RUN apt-get update && apt-get install -y lsb-release wget build-essential gettext

RUN echo "deb http://apt.postgresql.org/pub/repos/apt $(lsb_release -cs)-pgdg main" > /etc/apt/sources.list.d/pgdg.list
RUN wget --quiet -O - https://www.postgresql.org/media/keys/ACCC4CF8.asc | apt-key add -
RUN apt-get update && apt-get install -y postgresql-client-18 && rm -rf /var/lib/apt/lists/*

COPY requirements.txt /
RUN pip install --no-cache-dir -r /requirements.txt
WORKDIR /app

CMD ["python"]
```

web/requirements.txt

```
Django<6.0
psycopg2-binary==2.9.10
django-extensions==4.1
ipython==9.4.0
django-environ==0.12.0
requests==2.32.4
uwsgi
redis==5.2.1
```

Compose uWSGI + Django

```
feroda@paco:~/play-cloud-app$ cat compose-uwsgi.yml
services:

  web:
    build: ./web/.
    env_file: .env
    volumes:
      - ./web:/app
    command: uwsgi --ini uwsgi.ini
    stop_signal: SIGINT
    networks:
       - appnet
    restart: unless-stopped

networks:
      appnet:
```

Ora mettiamo insieme

- ✅ **Web Server NGINX**
- ✅ **Application Server uWSGI**
- ✅ **Server da database PostgreSQL**
- ✅ **Cache server REDIS**

In un unico compose.yml e per eseguirlo

docker-compose up -d

Una nota su NGINX

✅ **NGINX riceve la richiesta HTTP dal browser** e la instrada con il proprio file di configurazione

- ✅ **se sotto /static/**, restituisce i file statici dalla directory configurata
- ✅ **altrimenti** inoltra la richiesta a uWSGI, che esegue l'applicazione Python e genera la risposta

E il famoso terminatore SSL?

... in compose-ssl.yml!